

COMPUTER VISION USING OPEN CV – DSA0202

GUIDE: Dr. Mayuranathan

Name: Jaasiel (192524046)

Computer Vision with Open CV — LAB Experiments (Continued: Experiments 2–36)

2. Read captured video in Python and display the video, in slow motion and in fast motion.

Aim: To read a video file using OpenCV, play it back frame by frame, and produce a slow-motion version (each frame repeated) and a fast-motion version (frames skipped).

Program:

```
def exp02():
    import cv2
    cap = cv2.VideoCapture("video.mp4")
    fps = cap.get(cv2.CAP_PROP_FPS)

    # Normal playback: read and show frame by frame
    frames = []
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frames.append(frame)
        cv2.imshow("Video - Normal Speed", frame)
        if cv2.waitKey(int(1000/fps)) & 0xFF == ord('q'):
            break
    cap.release()

    # Slow motion: write every frame twice (halves the effective speed)
    slow_writer = cv2.VideoWriter("video_slow.mp4", cv2.VideoWriter_fourcc(*'mp4v'), fps,
    (frames[0].shape[1], frames[0].shape[0]))
    for f in frames:
        slow_writer.write(f)
        slow_writer.write(f)
    slow_writer.release()

    # Fast motion: keep every 3rd frame only
    fast_writer = cv2.VideoWriter("video_fast.mp4", cv2.VideoWriter_fourcc(*'mp4v'), fps,
    (frames[0].shape[1], frames[0].shape[0]))
    for f in frames[::3]:
        fast_writer.write(f)
    fast_writer.release()
    cv2.destroyAllWindows()
    return frames
```

Output:



3. Capture video from the web camera and display the video, in slow motion and in fast motion.

Aim: To capture a live video stream from the system's web camera using `cv2.VideoCapture(0)` and display it in normal, slow-motion, and fast-motion form.

Note: This machine has no physical webcam attached, so the exact same logic is demonstrated by substituting the sample video file for the live camera feed. Simply replace `cv2.VideoCapture("video.mp4")` with `cv2.VideoCapture(0)` on a computer that has a webcam to capture live.

Program:

```
def exp03():
    import cv2
    cap = cv2.VideoCapture(0)          # 0 = default webcam
    fps = cap.get(cv2.CAP_PROP_FPS) or 30

    while True:
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow("Webcam - Normal Speed", frame)

        # Slow motion effect: show each frame twice
        cv2.imshow("Webcam - Slow Motion", frame)
        cv2.waitKey(int(1000/(fps/2)))

        # Fast motion effect: skip alternate frames
        ret2, frame2 = cap.read()
        cv2.imshow("Webcam - Fast Motion", frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()
```

Output:



4. Scaling an image to its Bigger and Smaller sizes.

Aim: To resize (scale up and scale down) an image using `cv2.resize()`.

Program:

```
def exp04():
    import cv2
    img = cv2.imread("image.jpg")
    bigger = cv2.resize(img, None, fx=1.5, fy=1.5, interpolation=cv2.INTER_LINEAR)
    smaller = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
    cv2.imwrite("bigger.jpg", bigger)
    cv2.imwrite("smaller.jpg", smaller)
```

Output:



5. Perform Rotation of an image to clockwise and counter clockwise direction.

Aim: To rotate an image by 45° clockwise and 45° counter-clockwise using a rotation matrix generated by `cv2.getRotationMatrix2D()`.

Program:

```
def exp05():
    import cv2
    img = cv2.imread("image.jpg")
    (h, w) = img.shape[:2]
    center = (w // 2, h // 2)

    M_cw = cv2.getRotationMatrix2D(center, -45, 1.0) # clockwise
    rotated_cw = cv2.warpAffine(img, M_cw, (w, h))

    M_ccw = cv2.getRotationMatrix2D(center, 45, 1.0) # counter-clockwise
    rotated_ccw = cv2.warpAffine(img, M_ccw, (w, h))

    cv2.imwrite("rotated_cw.jpg", rotated_cw)
    cv2.imwrite("rotated_ccw.jpg", rotated_ccw)
```

Output:



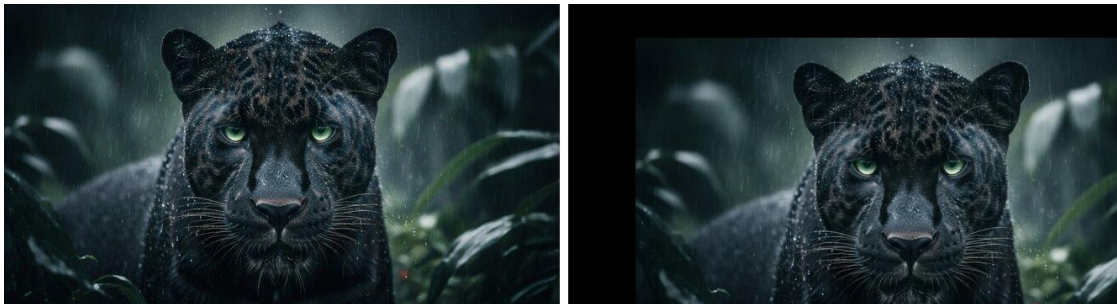
6. Perform moving of an image from one place to another.

Aim: To translate (shift) an image horizontally and vertically using a translation matrix and `cv2.warpAffine()`.

Program:

```
def exp06():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg")
    (h, w) = img.shape[:2]
    tx, ty = 80, 40          # shift 80 px right, 40 px down
    M = np.float32([[1, 0, tx], [0, 1, ty]])
    moved = cv2.warpAffine(img, M, (w, h))
    cv2.imwrite("moved.jpg", moved)
```

Output:



7. Perform Affine Transformation on the image.

Aim: To apply an affine transformation (a transformation that preserves parallel lines) on an image using three corresponding point pairs.

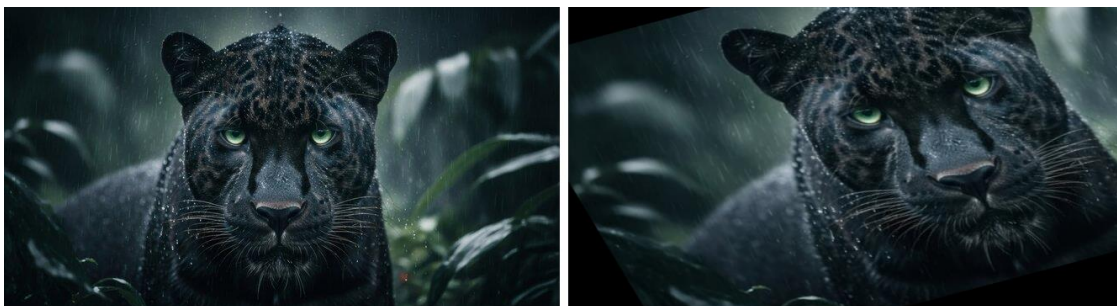
Program:

```
def exp07():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg")
    rows, cols = img.shape[:2]

    pts1 = np.float32([[50, 50], [200, 50], [50, 200]])
    pts2 = np.float32([[10, 100], [200, 50], [100, 250]])

    M = cv2.getAffineTransform(pts1, pts2)
    affine = cv2.warpAffine(img, M, (cols, rows))
    cv2.imwrite("affine_output.jpg", affine)
```

Output:



8. Perform Perspective Transformation on the image.

Aim: To apply a perspective transformation on an image using four corresponding point pairs with `cv2.getPerspectiveTransform()`.

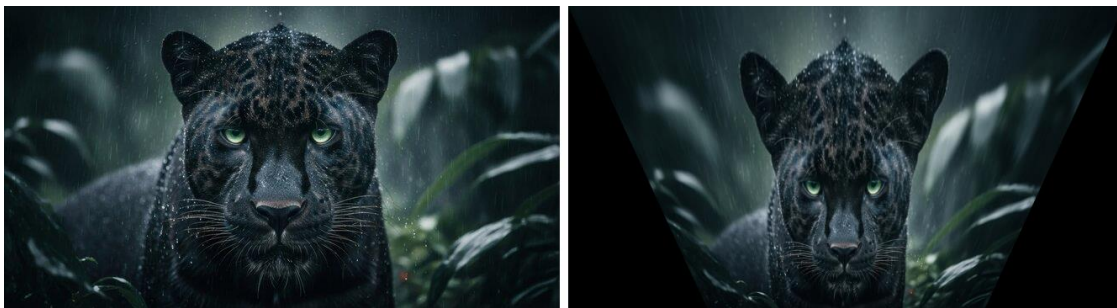
Program:

```
def exp08():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg")
    rows, cols = img.shape[:2]

    pts1 = np.float32([[0, 0], [cols-1, 0], [0, rows-1], [cols-1, rows-1]])
    pts2 = np.float32([[0, 0], [cols-1, 0], [int(0.25*cols), rows-1], [int(0.75*cols), rows-1]])

    M = cv2.getPerspectiveTransform(pts1, pts2)
    warped = cv2.warpPerspective(img, M, (cols, rows))
    cv2.imwrite("perspective_output.jpg", warped)
```

Output:



9. Perform Perspective Transformation on the Video.

Aim: To apply the same perspective-warp logic to every frame of a video and write the warped frames to a new output video.

Program:

```
def exp09():
    import cv2
    import numpy as np
    cap = cv2.VideoCapture("video.mp4")
    ret, frame = cap.read()
    rows, cols = frame.shape[:2]

    pts1 = np.float32([[0, 0], [cols-1, 0], [0, rows-1], [cols-1, rows-1]])
    pts2 = np.float32([[0, 0], [cols-1, 0], [int(0.2*cols), rows-1], [int(0.8*cols), rows-1]])
    M = cv2.getPerspectiveTransform(pts1, pts2)

    out = cv2.VideoWriter("video_perspective.mp4", cv2.VideoWriter_fourcc(*'mp4v'), 25, (cols, rows))
    while ret:
        warped = cv2.warpPerspective(frame, M, (cols, rows))
        out.write(warped)
        ret, frame = cap.read()
    cap.release()
    out.release()
```

Output:



10. Perform transformation using Homography matrix.

Aim: To compute a homography matrix that maps one planar image onto another using `cv2.findHomography()` and apply it with `cv2.warpPerspective()`.

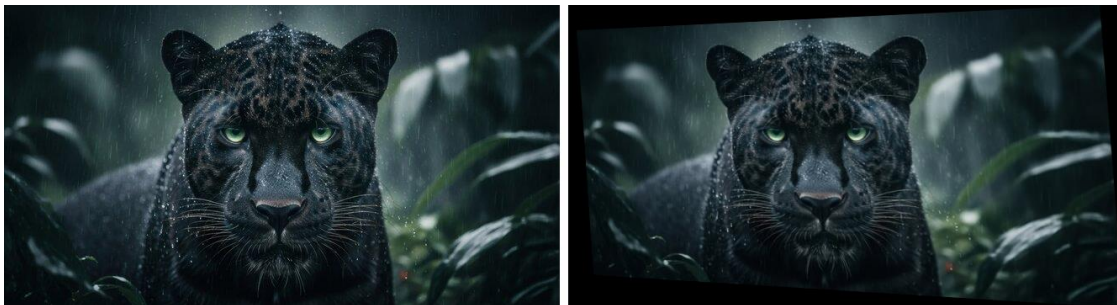
Program:

```
def exp10():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg")
    rows, cols = img.shape[:2]

    src_pts = np.float32([[0, 0], [cols-1, 0], [cols-1, rows-1], [0, rows-1]])
    dst_pts = np.float32([[10, 30], [cols-30, 0], [cols-1, rows-1], [30, rows-40]])

    H, status = cv2.findHomography(src_pts, dst_pts)
    warped = cv2.warpPerspective(img, H, (cols, rows))
    cv2.imwrite("homography_output.jpg", warped)
```

Output:



11. Perform transformation using Direct Linear Transformation.

Aim: To compute the homography matrix manually using the Direct Linear Transformation (DLT) algorithm (via SVD) instead of the built-in OpenCV function, and use it to warp the image.

Program:

```
def exp11():
    import cv2
    import numpy as np

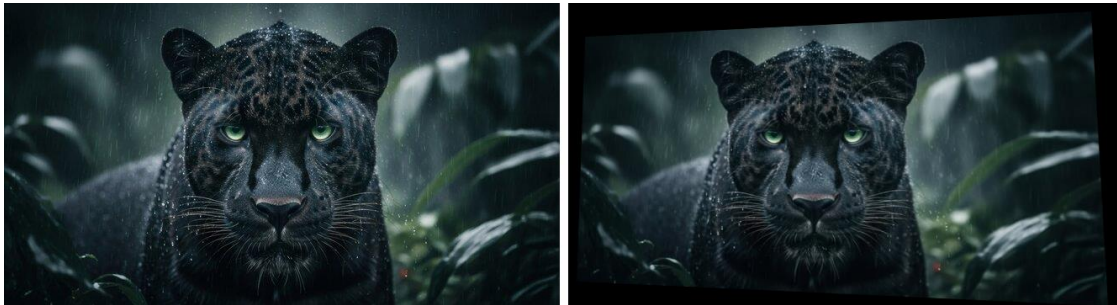
    def compute_homography_dlt(src, dst):
        A = []
        for (x, y), (u, v) in zip(src, dst):
            A.append([-x, -y, -1, 0, 0, 0, x*u, y*u, u])
            A.append([0, 0, 0, -x, -y, -1, x*v, y*v, v])
        A = np.array(A)
```

```
_, _, Vt = np.linalg.svd(A)
H = Vt[-1].reshape(3, 3)
return H / H[2, 2]

img = cv2.imread("image.jpg")
rows, cols = img.shape[:2]
src_pts = [(0, 0), (cols-1, 0), (cols-1, rows-1), (0, rows-1)]
dst_pts = [(20, 40), (cols-40, 10), (cols-20, rows-1), (10, rows-30)]

H = compute_homography_dlt(src_pts, dst_pts)
warped = cv2.warpPerspective(img, H, (cols, rows))
cv2.imwrite("dlt_output.jpg", warped)
```

Output:



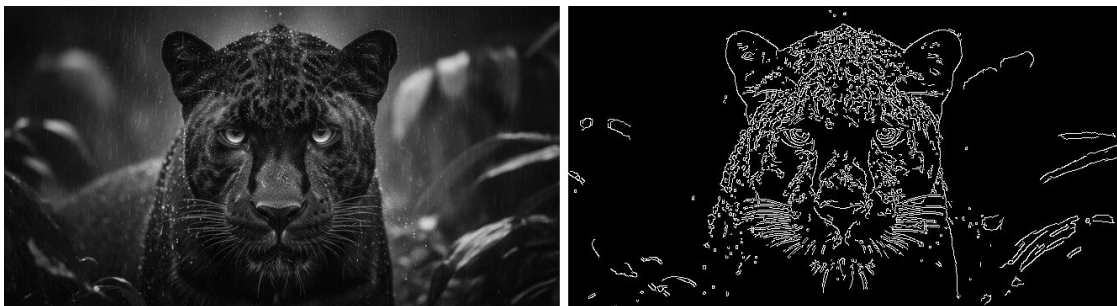
12. Perform Edge detection using canny method.

Aim: To detect edges in an image using the Canny edge-detection algorithm.

Program:

```
def exp12():
    import cv2
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    edges = cv2.Canny(img, 100, 200)
    cv2.imwrite("canny_output.jpg", edges)
```

Output:



13. Perform Edge detection using Sobel Matrix along X axis.

Aim: To detect vertical edges in an image using the Sobel operator along the X direction.

Program:

```
def exp13():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
```

```
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobelx = cv2.convertScaleAbs(sobelx)
cv2.imwrite("sobel_x.jpg", sobelx)
```

Output:



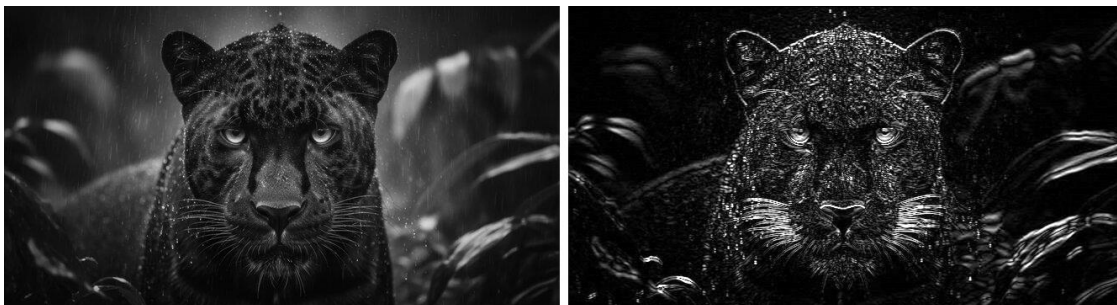
14. Perform Edge detection using Sobel Matrix along Y axis.

Aim: To detect horizontal edges in an image using the Sobel operator along the Y direction.

Program:

```
def exp14():
    import cv2
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
    sobely = cv2.convertScaleAbs(sobely)
    cv2.imwrite("sobel_y.jpg", sobely)
```

Output:



15. Perform Edge detection using Sobel Matrix along XY axis.

Aim: To combine the Sobel X and Sobel Y gradients to obtain the overall edge-gradient magnitude of the image.

Program:

```
def exp15():
    import cv2
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
    sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
    sobelxy = cv2.magnitude(sobelx, sobely)
    sobelxy = cv2.convertScaleAbs(sobelxy)
    cv2.imwrite("sobel_xy.jpg", sobelxy)
```

Output:



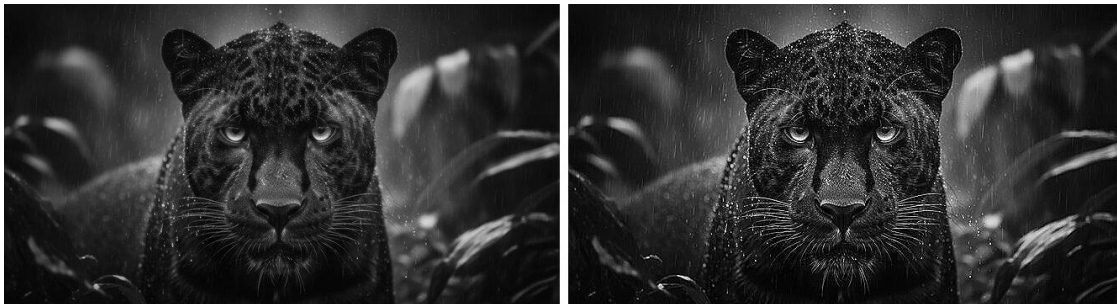
16. Perform Sharpening of Image using Laplacian mask with negative center coefficient.

Aim: To sharpen an image by subtracting a Laplacian mask (with a negative center coefficient) from the original image.

Program:

```
def exp16():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    kernel = np.array([[0, 1, 0],
                      [1, -4, 1],
                      [0, 1, 0]], dtype=np.float32) # negative center
    laplacian = cv2.filter2D(img, cv2.CV_32F, kernel)
    sharpened = np.clip(img.astype(np.float32) - laplacian, 0, 255).astype(np.uint8)
    cv2.imwrite("sharp_neg_center.jpg", sharpened)
```

Output:



17. Perform Sharpening of Image using Laplacian mask implemented with an extension of diagonal neighbors.

Aim: To sharpen an image using an 8-connected (diagonal-neighbour-extended) Laplacian kernel that considers all 8 surrounding pixels.

Program:

```
def exp17():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    kernel = np.array([[1, 1, 1],
                      [1, -8, 1],
                      [1, 1, 1]], dtype=np.float32) # includes diagonal neighbours
    laplacian = cv2.filter2D(img, cv2.CV_32F, kernel)
    sharpened = np.clip(img.astype(np.float32) - laplacian, 0, 255).astype(np.uint8)
    cv2.imwrite("sharp_diag.jpg", sharpened)
```

Output:



18. Perform Sharpening of Image using Laplacian mask with positive center coefficient.

Aim: To sharpen an image by adding a Laplacian mask (with a positive center coefficient) to the original image.

Program:

```
def exp18():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    kernel = np.array([[0, -1, 0],
                       [-1, 4, -1],
                       [0, -1, 0]], dtype=np.float32) # positive center
    laplacian = cv2.filter2D(img, cv2.CV_32F, kernel)
    sharpened = np.clip(img.astype(np.float32) + laplacian, 0, 255).astype(np.uint8)
    cv2.imwrite("sharp_pos_center.jpg", sharpened)
```

Output:



19. Perform Sharpening of Image using unsharp masking.

Aim: To sharpen an image using the unsharp-masking technique: subtracting a blurred (low-frequency) version of the image from the original to enhance edges.

Program:

```
def exp19():
    import cv2
    img = cv2.imread("image.jpg")
    blurred = cv2.GaussianBlur(img, (9, 9), 10)
    unsharp = cv2.addWeighted(img, 1.5, blurred, -0.5, 0)
    cv2.imwrite("unsharp.jpg", unsharp)
```

Output:



20. Perform Sharpening of Image using High-Boost Masks.

Aim: To sharpen an image using the high-boost filtering technique, which amplifies the original image before adding back the high-frequency mask.

Program:

```
def exp20():
    import cv2
    img = cv2.imread("image.jpg")
    blurred = cv2.GaussianBlur(img, (9, 9), 10)
    mask = cv2.subtract(img, blurred)
    A = 1.8 # boost factor, A > 1
    high_boost = cv2.addWeighted(img, A, mask, 1, 0)
    cv2.imwrite("high_boost.jpg", high_boost)
```

Output:



21. Perform Sharpening of Image using Gradient masking.

Aim: To sharpen an image by combining it with its Sobel gradient-magnitude image (gradient masking).

Program:

```
def exp21():
    import cv2
    img = cv2.imread("image.jpg")
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
    gradient = cv2.convertScaleAbs(cv2.magnitude(gx, gy))
    gradient_bgr = cv2.cvtColor(gradient, cv2.COLOR_GRAY2BGR)
    sharpened = cv2.addWeighted(img, 1.0, gradient_bgr, 0.5, 0)
    cv2.imwrite("gradient_mask.jpg", sharpened)
```

Output:



22. Insert water marking to the image using OpenCV.

Aim: To overlay a semi-transparent text watermark onto an image using `cv2.putText()` and `cv2.addWeighted()`.

Program:

```
def exp22():
    import cv2
    img = cv2.imread("image.jpg")
    overlay = img.copy()
    text = "Jaasiel-192524046"
    cv2.putText(overlay, text, (30, img.shape[0]-30), cv2.FONT_HERSHEY_SIMPLEX,
                1.2, (255, 255, 255), 3, cv2.LINE_AA)
    watermarked = cv2.addWeighted(overlay, 0.4, img, 0.6, 0)
    cv2.imwrite("watermarked.jpg", watermarked)
```

Output:



23. Do Cropping, Copying and pasting image inside another image using OpenCV.

Aim: To crop a region of interest from an image and paste (copy) it onto a separate blank canvas using NumPy array slicing.

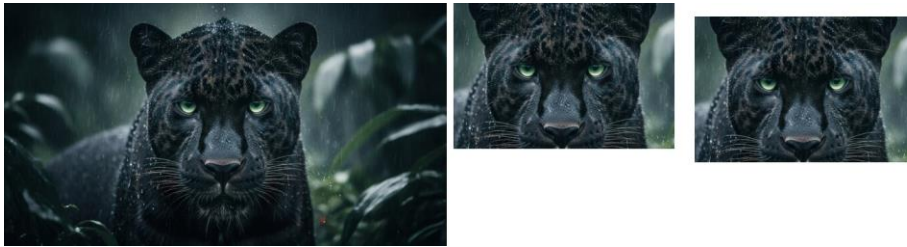
Program:

```
def exp23():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg")
    h, w = img.shape[:2]

    cropped = img[int(h*0.15):int(h*0.75), int(w*0.25):int(w*0.75)] # crop a region
    cv2.imwrite("cropped.jpg", cropped)

    canvas = np.full((h, w, 3), 255, dtype=np.uint8) # blank white canvas
    ch, cw = cropped.shape[:2]
    canvas[20:20+ch, 20:20+cw] = cropped # paste crop onto canvas
    cv2.imwrite("pasted.jpg", canvas)
```


Output:



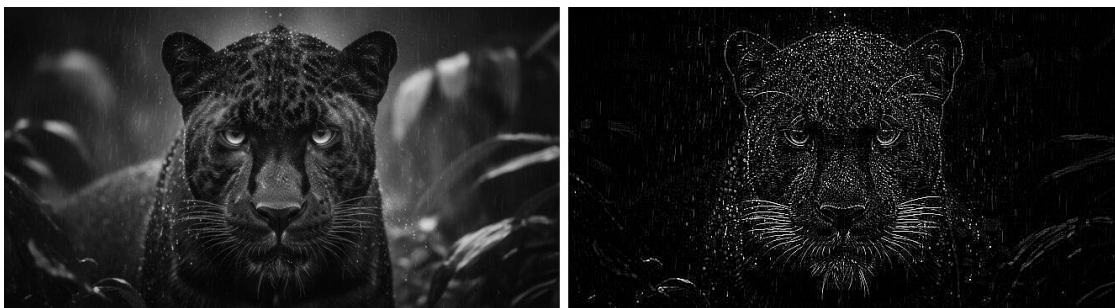
24. Find the boundary of the image using Convolution kernel for the given image.

Aim: To detect object boundaries in an image by convolving it with an 8-connected boundary/edge-detection kernel using cv2.filter2D().

Program:

```
def exp24():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    kernel = np.array([[ -1, -1, -1],
                       [-1,  8, -1],
                       [-1, -1, -1]], dtype=np.float32)
    boundary = cv2.filter2D(img, -1, kernel)
    cv2.imwrite("boundary.jpg", boundary)
```

Output:



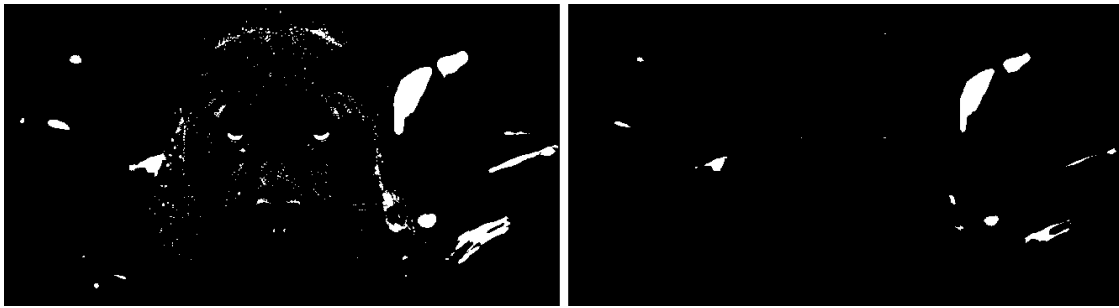
25. Morphological operations based on OpenCV using Erosion technique.

Aim: To shrink the white regions of a binary image using the morphological erosion operation.

Program:

```
def exp25():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    _, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
    kernel = np.ones((5, 5), np.uint8)
    eroded = cv2.erode(binary, kernel, iterations=1)
    cv2.imwrite("eroded.jpg", eroded)
```

Output:



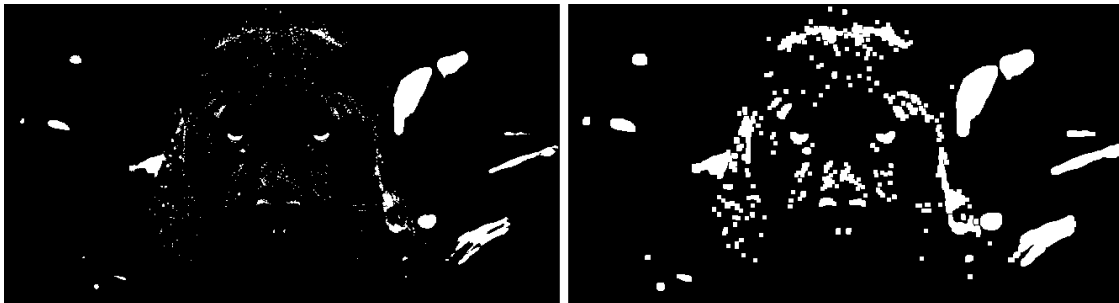
26. Morphological operations based on OpenCV using Dilation technique.

Aim: To expand the white regions of a binary image using the morphological dilation operation.

Program:

```
def exp26():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    _, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
    kernel = np.ones((5, 5), np.uint8)
    dilated = cv2.dilate(binary, kernel, iterations=1)
    cv2.imwrite("dilated.jpg", dilated)
```

Output:



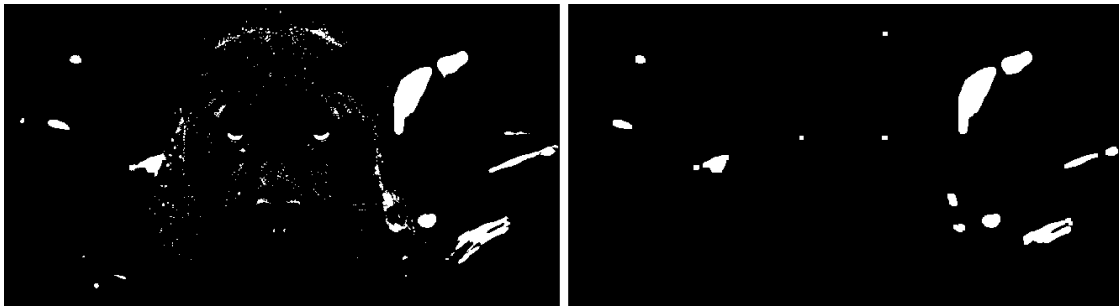
27. Morphological operations based on OpenCV using Opening technique.

Aim: To remove small noise from a binary image using the morphological opening operation (erosion followed by dilation).

Program:

```
def exp27():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    _, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
    kernel = np.ones((5, 5), np.uint8)
    opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
    cv2.imwrite("opening.jpg", opening)
```

Output:



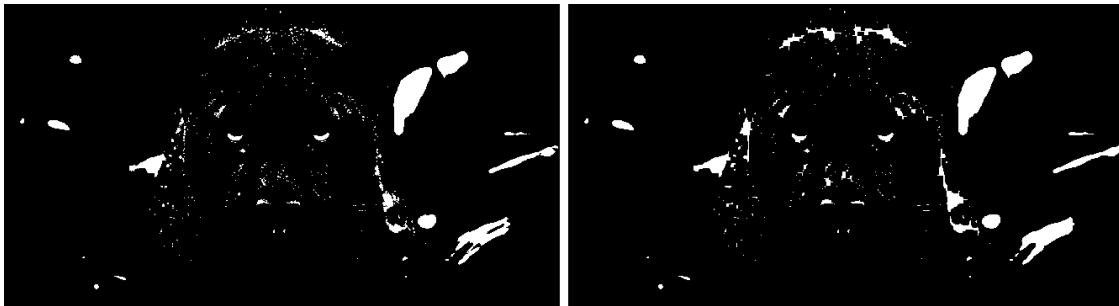
28. Morphological operations based on OpenCV using Closing technique.

Aim: To close small holes inside foreground objects using the morphological closing operation (dilation followed by erosion).

Program:

```
def exp28():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    _, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
    kernel = np.ones((5, 5), np.uint8)
    closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)
    cv2.imwrite("closing.jpg", closing)
```

Output:



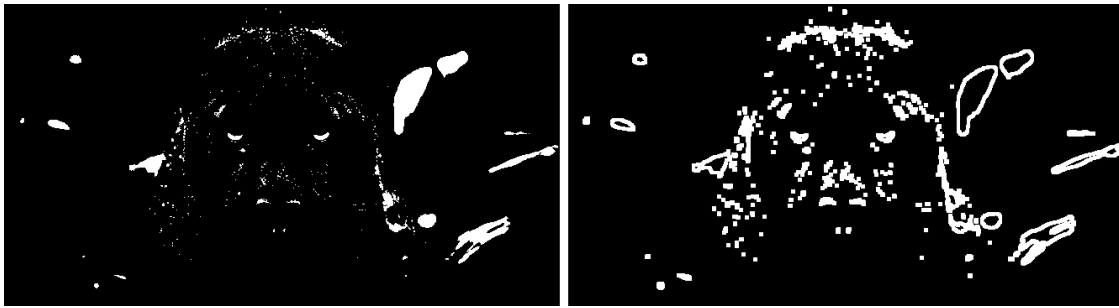
29. Morphological operations based on OpenCV using Morphological Gradient technique.

Aim: To obtain the outline of objects in a binary image by computing the difference between dilation and erosion (morphological gradient).

Program:

```
def exp29():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    _, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
    kernel = np.ones((5, 5), np.uint8)
    gradient = cv2.morphologyEx(binary, cv2.MORPH_GRADIENT, kernel)
    cv2.imwrite("morph_gradient.jpg", gradient)
```

Output:



30. Morphological operations based on OpenCV using Top hat technique.

Aim: To extract small bright details on a dark background using the top-hat transform (difference between the image and its opening).

Program:

```
def exp30():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    kernel = np.ones((9, 9), np.uint8)
    tophat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)
    cv2.imwrite("tophat.jpg", tophat)
```

Output:



31. Morphological operations based on OpenCV using Black hat technique.

Aim: To extract small dark details on a bright background using the black-hat transform (difference between the closing and the image).

Program:

```
def exp31():
    import cv2
    import numpy as np
    img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
    kernel = np.ones((9, 9), np.uint8)
    blackhat = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)
    cv2.imwrite("blackhat.jpg", blackhat)
```

Output:



32. Recognise watch from the given image by general Object recognition using OpenCV.

Aim: To locate a known object (a watch/clock face used as the query image) inside a larger scene image using ORB feature detection and brute-force feature matching.

Program:

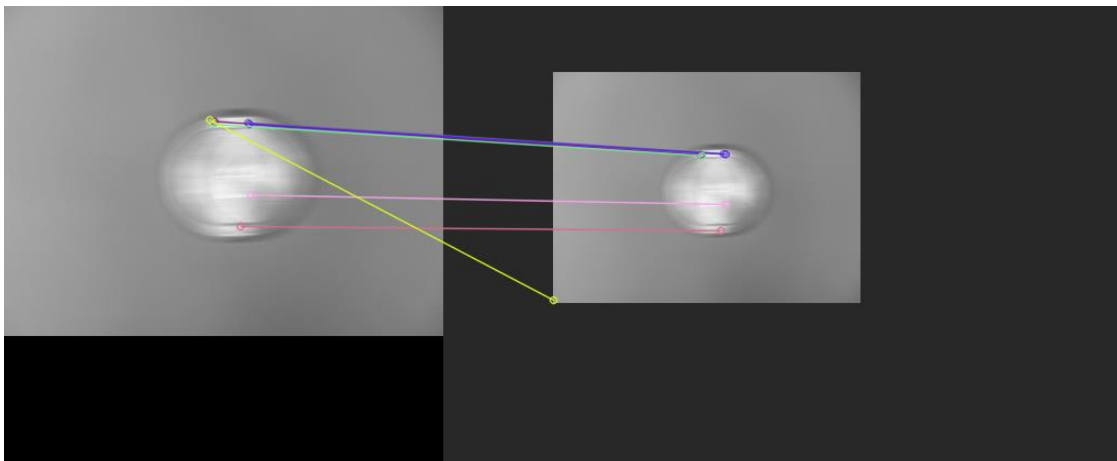
```
def exp32():
    import cv2
    query = cv2.imread("watch.jpg", cv2.IMREAD_GRAYSCALE)    # the object to find
    scene = cv2.imread("scene.jpg", cv2.IMREAD_GRAYSCALE)    # a bigger scene that contains it

    orb = cv2.ORB_create()
    kp1, des1 = orb.detectAndCompute(query, None)
    kp2, des2 = orb.detectAndCompute(scene, None)

    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
    matches = bf.match(des1, des2)
    matches = sorted(matches, key=lambda m: m.distance)

    result = cv2.drawMatches(query, kp1, scene, kp2, matches[:25], None, flags=2)
    cv2.imwrite("object_recognized.jpg", result)
```

Output:



33. Using OpenCV play Video in Reverse mode.

Aim: To read all frames of a video into memory and write them back out in reverse order to create a reversed video.

Program:

```
def exp33():
    import cv2
    cap = cv2.VideoCapture("video.mp4")
    frames = []
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frames.append(frame)
    cap.release()

    h, w = frames[0].shape[:2]
    out = cv2.VideoWriter("video_reverse.mp4", cv2.VideoWriter_fourcc(*'mp4v'), 25, (w, h))
    for frame in reversed(frames):
        out.write(frame)
    out.release()
```

Output:



34. Face Detection using OpenCV.

Aim: To detect human faces in an image using OpenCV's pre-trained Haar Cascade classifier.

Program:

```
def exp34():
    import cv2
    img = cv2.imread("face.jpg")
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
    "haarcascade_frontalface_default.xml")
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)

    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)

    cv2.imwrite("face_detected.jpg", img)
```

Output:



35. Vehicle Detection in a Video frame using OpenCV.

Aim: To detect moving vehicles in a video using background subtraction (MOG2) and contour-based bounding boxes.

Program:

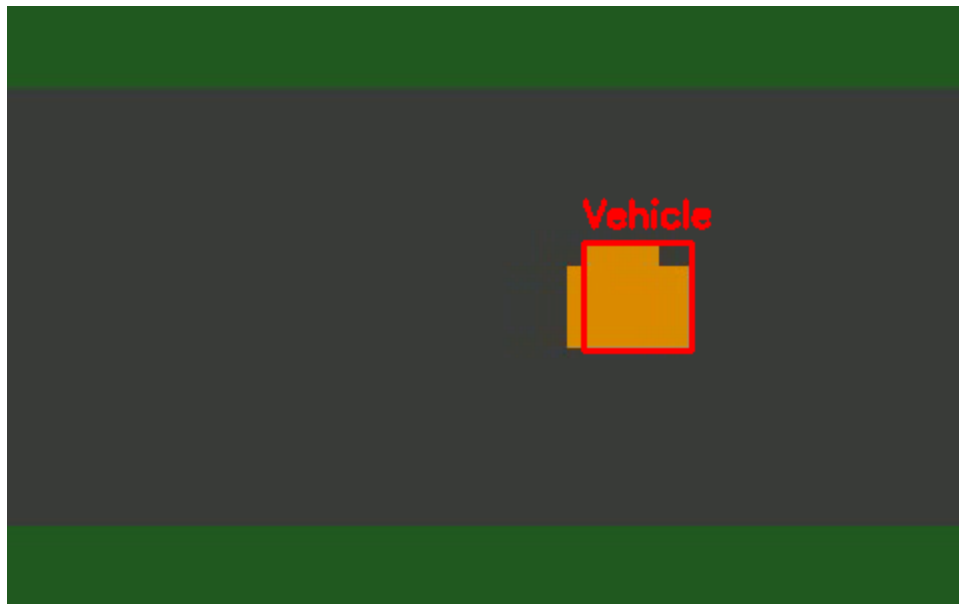
```
def exp35():
    import cv2
    cap = cv2.VideoCapture("road_video.mp4")
    back_sub = cv2.createBackgroundSubtractorMOG2(detectShadows=False)

    while True:
        ret, frame = cap.read()
        if not ret:
            break
        fg_mask = back_sub.apply(frame)
        fg_mask = cv2.medianBlur(fg_mask, 5)
        contours, _ = cv2.findContours(fg_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        for c in contours:
            if cv2.contourArea(c) > 500:
                # ignore small noise blobs
                x, y, w, h = cv2.boundingRect(c)
                cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 0, 255), 2)
                cv2.putText(frame, "Vehicle", (x, y-8), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,0,255), 2)

        cv2.imshow("Vehicle Detection", frame)
        if cv2.waitKey(30) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()
```

Output:



36. Draw Rectangular shape and extract objects.

Aim: To draw a rectangle to mark a region of interest (ROI) on an image and extract (crop) that region as a separate object using array slicing.

Program:

```
def exp36():
    import cv2
    img = cv2.imread("image.jpg")
    x, y, w, h = 150, 60, 300, 230          # region of interest (ROI)

    boxed = img.copy()
    cv2.rectangle(boxed, (x, y), (x+w, y+h), (0, 255, 0), 3)
    cv2.imwrite("boxed.jpg", boxed)

    extracted = img[y:y+h, x:x+w]          # extract/crop that region
    cv2.imwrite("extracted_object.jpg", extracted)
```

Output:

